

Aura Group: Policy-Bound Hybrid TreeKEM for Post-Quantum Group Messaging

Oleksandr Melnychenko

Khmelnytskyi National University, Khmelnytskyi, Ukraine

Abstract

AURA Group is the group-messaging protocol for AURA Messenger. It targets dynamic post-quantum confidential groups in which membership changes, application-message traffic, relay-blind delivery, and local state persistence belong to the same cryptographic state machine. The design is inspired by MLS and TreeKEM, but it is not wire-compatible with MLS.

Accepted group epochs advance through TreeKEM-style Commit messages whose update paths carry both X25519 and ML-KEM-768 material. Path secrets are encrypted to co-path resolutions with a hybrid X25519+ML-KEM-768 wrapping key, and every accepted Commit derives a group-epoch secret from the previous init secret, the fresh commit secret, and a context hash over the group identifier, epoch number, tree hash, and serialized security policy. Policy is therefore cryptographic input: disagreement about Shield, external joins, franking, sender-key lifetime, or skipped-key storage changes the accepted epoch state. Within an accepted epoch, per-member sender chains provide low-cost application messages while sender data, payloads, replay state, sealed content, expiring content, and franking commitments remain bound to the same epoch and policy.

The security account targets membership-change forward secrecy, post-compromise recovery at group-epoch boundaries, policy consensus, replay rejection, and accountable franking. Evidence is deliberately scoped: a ProVerif model discharges six queries for one accepted hybrid group epoch and one application message, while the reference implementation validates parsing, persistence, replay, external-join rejection, Shield enforcement, franking, sealed state, and benchmarked cost centers. Full asynchronous group security, multi-device state, delivery fairness, post-quantum signatures, and arbitrary proposal interleavings remain separate proof and deployment obligations.

Keywords: secure group messaging, TreeKEM, ML-KEM, message franking

1. Introduction

End-to-end encrypted group messaging protects a moving collective state, not only a sequence of pairwise messages. The central object is an accepted *group epoch*: a membership interval in which the roster, ratchet-tree hash, security policy, and application-secret schedule are fixed for all honest members. When membership changes, the protocol must move the group into a new epoch, deliver fresh secrets only to the new roster, prevent removed members from following future traffic, and keep ordinary application messages efficient between such state transitions.

AURA Group is the group-messaging protocol for AURA Messenger. The target setting includes ordinary private conversations, but also long-lived operational groups in which messages can carry inspection evidence, sensor alerts, incident decisions, or other records whose confidentiality matters after the immediate exchange has ended. In that setting, payload encryption alone is not the full security question. A relay must not become an authority over membership or content; a removed member must not learn later traffic; a re-added member must not recover messages from the absence interval; and stored ciphertext should remain protected against future post-quantum adversaries. This is the common threat shape behind coordinated operational communication: membership changes, evidence reporting, delayed review, and untrusted relay delivery occur in the same channel. Prior work on UAV trajectory adaptation, multimodal wind-turbine inspection, and UAV/SCADA-linked fire detection gives examples of domains with this coordination pattern [6, 7, 8]. Those applications are not protocol assumptions. They make concrete the broader requirement: membership, sender authentication, policy consensus, and post-quantum confidentiality must be treated as one cryptographic surface.

MLS and TreeKEM provide the closest standard vocabulary for this surface. An MLS-style Commit changes the authenticated group state, refreshes a path in the ratchet tree, distributes path secrets to co-path resolutions, and installs the next epoch after client-side verification [9, 10, 11]. The post-quantum migration problem is sharper in this setting than in a pairwise handshake. A group update that only refreshes classical DH material leaves the epoch transition dependent on assumptions that may not protect archived ciphertext against a future cryptographically relevant quantum computer. The terminology and parameter choices follow the NIST post-quantum standardization line and the ML-KEM specification used by

current hybrid deployments [12, 13].

AURA Group adopts the TreeKEM state-transition discipline but defines a compact hybrid protocol profile rather than an MLS wire-compatible extension. The technical question is specific: what changes when ML-KEM material is carried in TreeKEM update-path wrapping, and when serialized application security policy is included in the epoch context, key schedule, and confirmation input? Each hybrid update path carries both X25519 and ML-KEM-768 material, and the accepted group context binds the group identifier, epoch number, ratchet-tree hash, and serialized security policy. The post-quantum contribution is therefore part of the membership-changing tree update itself, not only an admission mechanism or a group-creation extension. Policy is also made cryptographic: a disagreement about Shield, external joins, franking, sender-key lifetime, or skipped-key storage changes the derived epoch state instead of remaining an application-level preference.

This design point is intentionally narrower than a new general group-security framework. The novelty is the protocol interface: hybrid path wrapping, policy-bound epoch derivation, efficient sender-chain traffic, and accountable message semantics are placed behind the same local acceptance boundary. That boundary is important because each mechanism alone is familiar; the security question is whether the accepted tree, accepted policy, sender-chain state, and message semantics move together without giving the relay authority over any of them.

The security account has a defined proof boundary. The specification and mechanized model focus on one accepted group epoch and on the transition mechanisms that lead into it. The accompanying implementation evidence checks parsing, persistence, replay handling, state rejection, and message semantics. A full concurrent-group theorem would require a separate tree-concurrency model covering all interleavings of Add, Remove, Update, PSK, and ReInit proposals.

Objective. This paper specifies and evaluates a dynamic group-messaging protocol that combines hybrid TreeKEM update paths, policy-bound epoch derivation, and efficient sender-chain traffic while keeping the proof, symbolic-model, and implementation-validation boundaries explicit enough for external review.

Contributions and novelty. AURA Group contributes a policy-bound hybrid TreeKEM profile in which post-quantum path refresh, policy consensus, sender-key economy, and epoch-bound message-layer hooks are specified as one group state machine. The contributions are as follows.

1. **G1: hybrid TreeKEM update paths.** Every epoch transition can refresh path material with both X25519 and ML-KEM-768 contributions.

The new path secrets are delivered to co-path resolutions through hybrid wrapping keys, so post-quantum material participates in ordinary membership-changing updates.

2. **G2: policy-bound group context.** The serialized security policy is hashed into the group context used for key derivation and confirmation. Silent disagreement about Shield, external joins, franking, sender-key lifetime, or skipped-key limits therefore changes the epoch keys rather than producing an apparently valid downgrade.
3. **G3: cryptographic Shield profile.** Shield is defined as a protocol profile rather than an advisory product mode. It selects profile-specific subkey labels, mandatory franking, external-join blocking, bounded skipped-key storage, and bounded sender-key lifetime as inputs to the accepted epoch state.
4. **G4: sender-chain traffic inside an accepted group epoch.** Per-member sender chains provide low-cost application messages between TreeKEM Commit messages, while encrypted sender data, replay windows, and epoch binding prevent duplicate or cross-interval acceptance.
5. **G5: epoch-bound message-layer hooks.** Franking, sealed content, and disappearing-message checks are bound to the accepted epoch and policy. They are not independent security claims; they specify where higher-level message semantics are checked after cryptographic decryption. The relay observes only structural metadata and opaque commitments.
6. **G6: scoped mechanized evidence.** A dedicated ProVerif model checks six obligations for one accepted group epoch: hybrid path secrecy, previous-init contribution secrecy, application-message secrecy, Commit correspondence, message correspondence, and franking correspondence. The scope is stated explicitly, so symbolic evidence is not confused with a full proof for arbitrary concurrent group schedules.

Table 1: Contribution-to-evidence map for the AURA Group scope.

ID	Group claim	Main evidence in this paper
G1	Hybrid update paths carry post-quantum material into ordinary group-epoch changes.	Update-path geometry (Figure 3), hybrid wrapper equations, recovery claim, and implementation validation of path processing (Table 9).
G2	Policy bytes are part of the accepted epoch, not advisory application metadata.	Group-context definition (Equation (1)), epoch key schedule (Equation (5)), policy-consensus claim, and Shield flow (Figure 4).
G3	Shield selects stricter processing rules through explicit protocol inputs.	Shield-mode construction, relay/member split (Table 5), external-join policy handling, and validation evidence for policy rejection.
G4	Sender chains keep within-epoch traffic efficient without erasing the epoch boundary.	Message algorithm (Algorithm 4), sender-chain derivation, replay claim, and message-layer validation.
G5	Franking, sealed content, and expiration are checked at accepted message boundaries.	Message-envelope figure (Figure 6), franking figure (Figure 5), franking claim, and ProVerif query Q6.
G6	Symbolic and executable evidence support a stated scope rather than a complete group proof.	Claim-boundary table (Table 2), verification-scope table (Table 7), ProVerif results (Table 8), and reproducibility notes.

2. Related Work and Design Position

2.1. MLS and TreeKEM

MLS is the closest standard reference point for the group-state problem. It separates group membership, group-epoch transitions, authenticated group context, and application-secret derivation into a protocol whose main state transition is a TreeKEM Commit message [9]. Here, a *Commit* means the MLS/TreeKEM protocol message that applies authenticated Add, Remove, Update, PSK, or ReInit proposals and installs the next group epoch after client-side verification. It is a group-state transition message, not an implementation or repository action. The TreeKEM line of work gives the asynchronous tree structure that makes large dynamic groups practical: a member does not need to establish a fresh pairwise secret with every other member for every update, because refreshed path secrets can be encrypted to co-path resolutions [10]. TreeSync then sharpens the engineering problem by treating authenticated tree synchronization as a first-class security

boundary rather than a mere serialization detail [11].

AURA Group follows this MLS/TreeKEM shape while defining its own protocol profile rather than an RFC 9420 wire-compatible variant. The design isolates a technical question central to Aura Messenger’s group architecture: how do hybrid classical/post-quantum update paths and policy-bound group-epoch derivation affect group-state evolution? This restriction is deliberate. The paper does not try to replace the MLS standard, and it does not claim the full security theorem expected for all MLS proposal schedules. It examines a smaller interface in which the post-quantum contribution is injected at the membership-changing update path and the application-security policy is bound before confirmation. That keeps the analyzed cryptographic surface tractable while preserving the group-specific problems that do not appear in a two-party ratchet.

2.2. Sender-Key Messaging and Message Semantics

Many deployed group messengers use sender-key style techniques for efficient traffic within a group epoch. A sender key avoids running a full group key update for every application message, but it shifts attention to per-sender counters, replay windows, skipped-key storage, sender authentication, and group-epoch separation. AURA Group adopts this efficiency pattern within a group epoch, but does not treat it as an independent protocol. Sender chains are derived from the accepted group-epoch state, and encrypted sender data is tied to the same metadata key that comes from the policy-bound key schedule.

The message-semantics layer is similarly integrated. Message franking provides selective accountable disclosure without giving the relay plaintext access to ordinary traffic [17]. Sealed content follows the same design instinct as sealed-sender metadata minimization: expose only what routing and delivery need, and keep content-level secrets inside end-to-end encrypted payloads [18]. In AURA Group these mechanisms are not treated as product features attached after encryption. They are explicit application-frame semantics whose tags, timestamps, and references are checked at the cryptographic processing boundary.

2.3. Post-Quantum Migration Boundary

The post-quantum migration pressure is different for pairwise handshakes and group-epoch intervals. A pairwise protocol can add a KEM encapsulation to the initial setup or to directed ratchet steps. A group protocol must decide how KEM material enters a tree update, how the corresponding ciphertexts are addressed to a co-path resolution, and how the accepted group-epoch interval records that policy and tree state agreed. AURA Group therefore moves ML-KEM material

into the TreeKEM update path rather than limiting it to account admission or group creation.

The design remains conservative about authentication. Ed25519 is used for key packages, Commit messages, and message signatures. That is a classical authentication boundary and is stated as a limitation rather than hidden behind the confidentiality claims. The contribution here is hybrid group-epoch confidentiality and policy agreement for group state; replacing Ed25519 with a post-quantum signature scheme is a separate design and deployment decision.

2.4. Two-Party Aura Protocol and Account Admission

The two-party AURA Protocol manuscript studies a pairwise handshake and directed ratchet boundary [1]. The Hybrid PQ-OPAQUE manuscript and its formal-evidence report study password-authenticated account admission in the augmented-PAKE tradition of Jarecki–Krawczyk–Xu and the IRTF OPAQUE specification [2, 3, 4, 5]. AURA Group uses those layers as surrounding protocol context, not as substitutes for group-security analysis. After credentials and identities are authenticated, the group protocol must still define accepted tree state, epoch transitions, sender-chain traffic, replay control, and relay-visible encrypted envelopes.

3. Threat Model and Scope

The adversary controls the network, reorders and replays messages, corrupts old serialized state, tampers with Commit messages and application ciphertexts, and may learn secrets of removed members. The relay is not trusted with plaintext or group secrets. It may validate structure and route messages, but authoritative membership and accepted group-epoch state are client-side decisions.

The model is single-device. Identity authentication uses Ed25519 signatures over key packages and Commit messages. This is classical authentication; a transition to post-quantum signatures is orthogonal to the TreeKEM and policy-binding mechanisms. The security account combines a scoped ProVerif model, reference-implementation evidence, and a protocol argument. Complete theorems for asynchronous concurrent groups, including delivery schedules and all interleavings of Add, Remove, Update, PSK, and ReInit proposals, are outside this model.

The adversary may be adaptive at the level of endpoint state snapshots. A snapshot can reveal stored group-epoch secrets, sender-chain state, replay-window state, pending proposals, and serialized group state that was not sealed or has already been restored. The model still assumes correct local cryptographic erasure after

successful group-epoch processing: a member that keeps obsolete path secrets, group-epoch secrets, or message keys indefinitely has stepped outside the claim boundary. This is the same practical boundary that appears in pairwise ratchet protocols, but group messaging makes the state surface larger because there are per-sender chains and tree nodes in addition to a single pairwise root key.

The relay is fully adversarial for confidentiality and membership authority. It may drop a Commit message, delay a Welcome, replay an old message, route one member through stale public state, or present different clients with different delivery schedules. Clients therefore accept only local state transitions that verify the relevant signatures, tree hashes, policy bytes, confirmation MACs, and replay windows. The relay can still perform useful structural checks, including size limits and sender binding checks, but those checks are not a substitute for client-side cryptographic acceptance.

3.1. Security Goals

The construction is organized around six goals. First, accepted group-epoch secrets are intended to be pseudorandom to an adversary that did not learn both the surviving hybrid path material and the previous init contribution. Second, an honest post-compromise update creates a recovery boundary at the next accepted group epoch, provided at least one hybrid component remains unbroken and old secrets are erased. Third, removed or not-yet-added members are excluded from messages outside their membership interval. Fourth, members that disagree on policy fail to share a group epoch silently. Fifth, application messages reject replay inside the accepted group epoch. Sixth, frankable messages expose only selective disclosures to a moderator.

4. Construction Overview

AURA Group has three layers. The group-epoch layer maintains a ratchet tree and advances it with Commit messages. The key-schedule layer derives group-epoch secrets, metadata keys, welcome keys, confirmation keys, and sender-key bases. The message layer encrypts application payloads under per-sender chains while encrypting sender metadata under the group-epoch metadata key.

The tree is left-balanced and stores both X25519 and ML-KEM-768 public material at leaves and internal nodes [14, 13]. A member's key package binds identity Ed25519, identity X25519, identity ML-KEM-768, leaf X25519, leaf ML-KEM-768, and credential bytes under an Ed25519 signature. This separates long-

Table 2: Claim boundary for the group paper.

Topic	Claimed here	Not claimed here
Group-epoch secrecy	Fresh group-epoch keys after accepted hybrid TreeKEM Commit messages, subject to at least one unbroken component and honest erasure.	Universal-composability proof for all concurrent group schedules.
Post-compromise recovery	Recovery at group-epoch boundaries when an honest update path introduces fresh X25519 and ML-KEM-768 material after compromise.	Recovery without a later honest group-epoch update.
Policy enforcement	Policy bytes are hashed into the group context and affect group-epoch keys and confirmation MACs.	Server-side policy authority or mutable policy negotiation.
Relay role	Relay-blind structural validation and routing.	Relay knowledge of plaintext, group secrets, or authoritative membership state.
Application semantics	Franking, sealed content, and expiration are enforced at message processing boundaries.	Legal moderation semantics beyond cryptographic verification of a disclosed message.

term identity from group-epoch update material while giving recipients a signed object for membership admission.

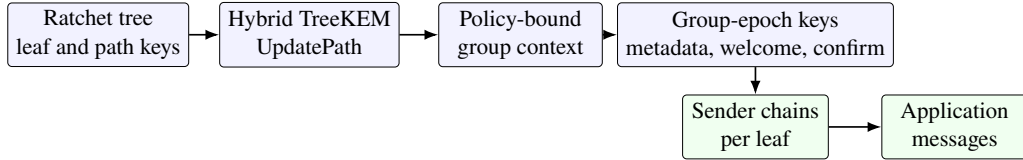


Figure 1: Group-epoch transition and message-key flow. A Commit message updates the tree, derives a policy-bound context, and refreshes sender chains for the new membership interval.

4.1. Key Packages and Welcome

Adding a member produces a signed key package, an Add Commit message, and a Welcome message. The Welcome encrypts the joiner secret for the added member and carries enough public group information for the recipient to reconstruct the tree, verify the tree hash, parse the serialized policy, derive group-epoch keys, and check the confirmation MAC. The design therefore keeps admission state

explicit: a member joins after decrypting Welcome and accepting the resulting group-epoch state, not because the relay appended an identity to a roster.

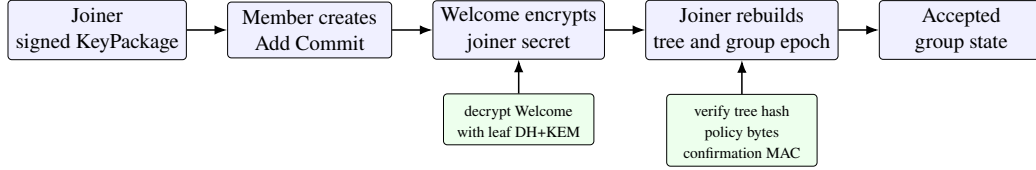


Figure 2: Welcome/admission boundary. A new member becomes authoritative only after reconstructing the tree, parsing the policy, deriving the group-epoch keys, and verifying the confirmation MAC.

Table 3: Group-state objects and their role.

Object	Contents	Role in the construction
Key package	Identity keys, leaf update keys, credential, signature.	Authenticated input for adding a member to the tree.
UpdatePath	Leaf public keys, path node public keys, encrypted path secrets, parent hashes.	Refreshes group-epoch tree material for current members.
Group context hash	Group identifier, group-epoch number, tree hash, serialized policy.	Binds key derivation to tree state and security policy.
Welcome	Encrypted joiner secret and public group information.	Allows a newly added member to reconstruct and verify group-epoch state.
Application message	Encrypted sender data, encrypted payload, nonces, signature, optional franking tag.	Carries traffic within a group epoch under sender-chain keys.

4.2. Tree Path Geometry

The construction follows the TreeKEM intuition that a member refreshes the nodes on its direct path and encrypts the corresponding path secrets to the co-path resolution. Figure 3 shows the case of leaf 2 in an eight-member tree. The shaded path is recomputed by the committer; the marked co-path nodes are sufficient recipients for the encrypted path secrets. The figure also shows the operational effect of the Commit: fresh path entropy is expanded along the direct path, wrapped to the co-path resolution, and then used as the commit secret for the next group epoch.

4.3. Hybrid Update Path

For a Commit at group epoch e , the committer samples a fresh path secret and derives node secrets along its direct path. Each updated node receives a fresh

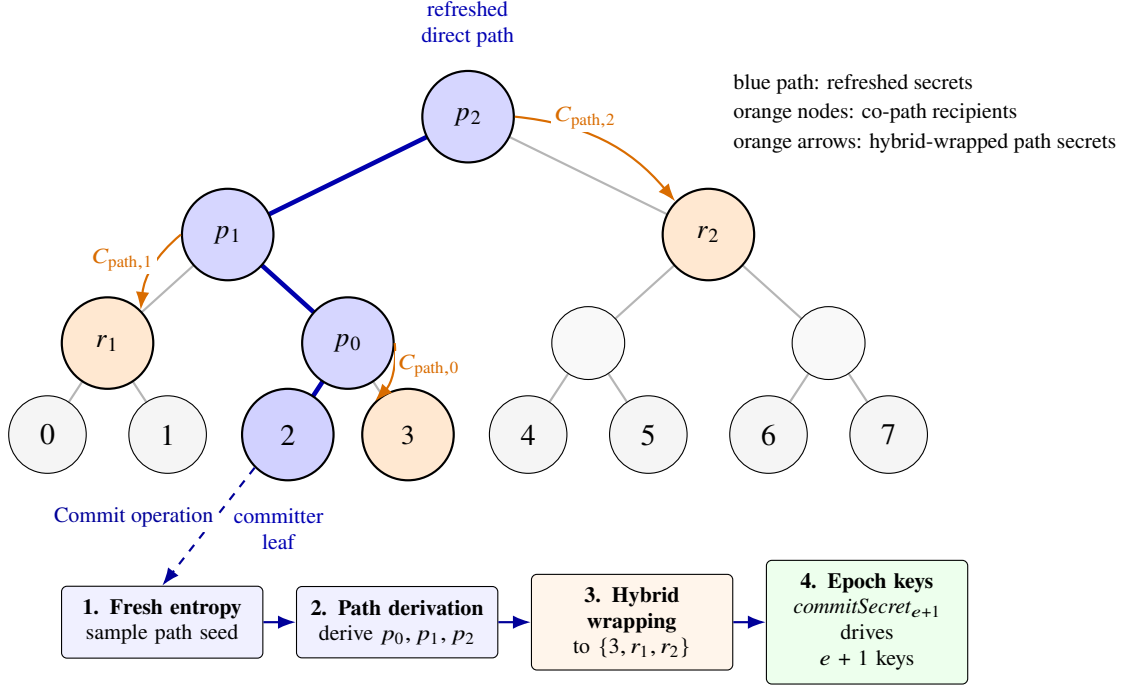


Figure 3: Core Hybrid TreeKEM UpdatePath and group-epoch effect for a Commit from leaf 2. The committer refreshes its direct path, encrypts the new path secrets to the co-path resolution, and feeds the recovered commit secret into the next group-epoch key schedule.

X25519 keypair and a fresh ML-KEM-768 keypair. In this profile, Add, Remove, and Update transitions that refresh tree secrets carry an UpdatePath; PSK and ReInit transitions still advance the epoch state but are not counted in the one-step hybrid path-secrecy argument unless they also carry fresh path material. A co-path resolution is the TreeKEM recipient set covering sibling subtrees that need the refreshed path secret in order to reconstruct the new tree state. For every recipient resolution node R , the path secret is wrapped under a hybrid key:

$$\begin{aligned}
(x_{\text{eph}}, X_{\text{eph}}) &\leftarrow \text{X25519.KeyGen}(), \\
z_{\text{X25519}} &= \text{X25519}(x_{\text{eph}}, X_R), \\
(c_{\text{ML-KEM-768}}, z_{\text{ML-KEM-768}}) &\leftarrow \text{ML-KEM-768.Encaps}(pk_R), \\
s_{\text{path}} &= \text{"Aura-PQ-Group-Hybrid"} \parallel z_{\text{X25519}}, \\
i_{\text{path}} &= \text{"Aura-Group-HybridPath"}, \\
k_{\text{path}} &= \text{HKDF}(z_{\text{ML-KEM-768}}, 32, s_{\text{path}}, i_{\text{path}}), \\
\text{aad}_{\text{path}} &= \text{gid} \parallel e \parallel \text{nodeIndex} \parallel X_{\text{eph}} \parallel c_{\text{ML-KEM-768}}, \\
C_{\text{path}} &= \text{AES-256-GCM-SIV.Enc}(k_{\text{path}}, n_{\text{path}}, \text{pathSecret}, \text{aad}_{\text{path}}).
\end{aligned}$$

Here $\text{HKDF}(x, L, s, i)$ denotes HKDF-SHA256 with input keying material x , output length L , salt s , and domain-separation information i . This matches the construction’s hybrid path wrapper: the ML-KEM-768 shared secret is the input material, and the X25519 output is carried in the salt with the path-domain label. The corresponding recipient decapsulates $c_{\text{ML-KEM-768}}$, computes the matching X25519 value, reconstructs k_{path} , and decrypts the path secret. The group identifier, group-epoch number, node index, ephemeral X25519 public key, and ML-KEM-768 ciphertext are associated data, so a wrapped path secret cannot be moved to a different group, group-epoch interval, tree position, or hybrid wrapper without detection. Parent hashes are verified along the updated path so that an adversary cannot splice a valid node into an inconsistent tree transcript. HKDF and the AEAD interface follow the standard extract-then-expand and nonce-misuse-resistant AEAD references [15, 16].

4.4. Policy-Bound Group-Epoch Context

A *group epoch* in AURA Group is a cryptographic membership interval: one accepted ratchet tree, one serialized policy, one group-epoch secret, one metadata key, and one set of per-member sender chains. Group epoch 0 is the creator-only genesis interval. Every accepted Add, Remove, Update, PSK, or ReInit Commit message advances the group from e to $e + 1$ atomically; application messages only consume sender-chain generations inside the current interval, and a rejected Commit leaves local state unchanged. The transition occurs only at an authenticated state boundary: the accepted TreeKEM Commit message, where tree hash, policy bytes, confirmation MAC, and group-epoch number agree before new keys are installed.

The terminology is fixed as follows. The *epoch number* e is the monotonic counter carried in group messages and Commit messages. The *group context hash* gch_e is the hash of the group identifier, epoch number, tree hash, and serialized policy. The *epoch secret* $epochSecret_e$ is derived only after an accepted transition into epoch e . The *accepted epoch state* is the local installed tuple consisting of the tree, policy, context hash, epoch keys, sender-chain bases, replay state, and sealed-state freshness metadata. Below, “group epoch” denotes the membership interval, and “accepted epoch state” denotes the installed local state.

The group-epoch context is defined with length prefixes for variable-size fields:

$$gch_e = \text{SHA-256}(\ell_{32}(gid) \parallel gid \parallel \ell_{64}(e) \parallel \ell_{32}(treeHash_e) \parallel treeHash_e \parallel \ell_{32}(policy) \parallel policy). \quad (1)$$

Here $\ell_{32}(x)$ is the 32-bit little-endian length of x , and $\text{le}_{64}(e)$ is the 64-bit little-endian group-epoch encoding. The length prefixes remove concatenation ambiguity. More importantly, the serialized policy is inside the hash. If two members disagree on whether Shield is enabled, or on the sender-key limits, they compute different gch_e , different group-epoch keys, and different confirmation MACs. For compact pseudocode, write $\text{GroupContext}(gid, e, treeHash, policy)$ for the hash in Equation (1).

For an UpdatePath-bearing accepted transition into group epoch e , let commitSecret_e be the fresh secret recovered from the accepted UpdatePath and let initSecret_{e-1} be the chaining secret carried from the previous group epoch. PSK-only or ReInit-only transitions may still advance the epoch state through their own transition secret, but they are outside the one-step hybrid path-secrecy argument unless they also carry fresh path material. The key schedule first combines continuity with transition entropy and then binds the result to the policy-bound group context:

$$\mathcal{A} = \{\text{meta}, \text{welcome}, \text{confirm}, \text{init}\}.$$

$$j_e = \text{HKDF-Extract}(\text{initSecret}_{e-1}, \text{commitSecret}_e), \quad (2)$$

$$\text{epochSecret}_e = \text{HKDF-Expand}(j_e, 32, \text{"Aura-Group-EpochSecret"} \parallel gch_e), \quad (3)$$

$$k_e^\alpha = \text{KDF}_\alpha^{\text{policy}}(\text{epochSecret}_e), \quad \alpha \in \mathcal{A}, \quad (4)$$

$$\mathbf{k}_e := (k_e^{\text{meta}}, k_e^{\text{welcome}}, k_e^{\text{confirm}}, \text{initSecret}_e). \quad (5)$$

Here $\text{KDF}_\alpha^{\text{policy}}$ is selected by the serialized policy. For the default profile,

$$\text{KDF}_\alpha^{\text{policy}}(x) = \text{HKDF-Expand}(x, 32, \text{label}(\alpha)).$$

The default label map is:

$$\begin{aligned} \text{label}(\text{meta}) &= \text{"Aura-Group-MetadataKey"}, \\ \text{label}(\text{welcome}) &= \text{"Aura-Group-WelcomeKey"}, \\ \text{label}(\text{confirm}) &= \text{"Aura-Group-ConfirmKey"}, \\ \text{label}(\text{init}) &= \text{"Aura-Group-InitSecret"}. \end{aligned}$$

Write $\text{Subkeys}_{\text{policy}}(x)$ for the tuple $(k^{\text{meta}}, k^{\text{welcome}}, k^{\text{confirm}}, \text{initSecret})$ derived by these policy-selected KDFs.

The four outputs have separate protocol roles. k_e^{meta} encrypts sender data, k_e^{welcome} domain-separates Welcome-admission material and supplemental Welcome fields, k_e^{confirm} authenticates the accepted group context, and initSecret_e is

the chaining secret for the next group-epoch transition. The extract output j_e is the group-epoch joiner secret; in Add transitions the core joiner secret is sealed to the joining member's key package inside a Welcome rather than encrypted under a group-wide key. A recipient recomputes the group-epoch secret $epochSecret_e$ only after reconstructing the same $(gid, e, treeHash_e, policy)$ context. The corresponding confirmation tag is:

$$\tau_e = \text{HMAC}(k_e^{\text{confirm}}, gch_e). \quad (6)$$

Thus the confirmation step checks tree state and policy state together.

4.5. Group-Epoch Installation and State Continuity

Group-epoch installation is transactional. A client first builds a candidate tree, candidate group context, candidate group-epoch secret, candidate subkeys, and candidate sender-chain bases. None of these candidate values become live state until the Commit signature, proposal validity, parent hashes, context binding, and confirmation MAC have all verified. For malformed input or a failed cryptographic check, the state transition is pure rejection:

$$\text{ProcessCommit}(S, \text{com}) = (\perp, S). \quad (7)$$

For accepted Commit messages, the abstract transition is a single step $S_e \rightarrow S_{e+1}$, even if an implementation parses, decrypts, derives, persists, notifies callbacks, and erases temporary secrets in separate instructions. Plaintext or membership callbacks should be released only after the corresponding replay state, group-epoch counter, sender-chain state, and sealed state have reached an equivalent durable boundary:

$$\text{Accept}(S_e, \text{com}) = S_{e+1} \implies \text{Persist}(S_{e+1}) < \text{Expose}(\text{effect}). \quad (8)$$

This is not an additional cryptographic assumption; it is the state-continuity condition needed for the replay and membership claims to hold. A crash between plaintext release and replay-window persistence would violate the protocol state premise even if all AEAD tags and signatures were valid.

4.6. Shield Mode

Shield is a fixed stricter policy profile. Its default values are profile parameters rather than theorem parameters: a 1000-generation limit per sender chain within a group epoch, 4 skipped sender keys per sender, external joins blocked, profile-specific key schedule enabled, and mandatory franking enabled. The bounds are

Table 4: Group-epoch state transitions and non-transitions.

Event	Effect on group-epoch state	Security consequence
Genesis creation	Creates group epoch 0 from the initial tree, policy, and group context.	Establishes the first authoritative policy-bound state.
Add, Remove, Update, PSK, ReInit Commit message	Advances e to $e + 1$ only after signature, proposal, applicable path, parent-hash, context, and confirmation checks.	Refreshes tree secrets when path material is present, resets sender-chain bases, and defines the next membership interval.
Application message	Does not advance the group-epoch counter. It consumes a sender-chain generation inside the current membership interval.	Preserves low-cost traffic while keeping replay state sender-indexed and group-epoch-bound.
Delayed message from a previous group epoch	May be accepted only from bounded compatibility state tagged by group epoch.	Does not reinstall an old tree or revive old Commit authority.
Rejected Commit message or ciphertext	Leaves the live group epoch, tree, sender chains, replay windows, and sealed-state freshness unchanged.	Prevents malformed input from creating partial state transitions.
Re-addition after removal	Requires a later Add or valid external join that creates a new membership interval.	Does not grant access to messages from the absence interval.

chosen to limit sender-chain lifetime and skipped-key storage in high-risk groups; the security statements depend on enforcement of the bounds, not on the exact numerical constants. The Shield subkey schedule uses two HKDF-Expand passes with distinct labels:

$$t_\alpha = \text{HKDF-Expand}(\text{epochSecret}_e, 32, \text{label}(\alpha) \parallel \text{"Aura-Enhanced-Pass1"}), \quad (9)$$

$$k_e^\alpha = \text{HKDF-Expand}(t_\alpha, 32, \text{label}(\alpha) \parallel \text{"Aura-Enhanced-Pass2"}). \quad (10)$$

The second pass is not a claim of extra strength beyond HKDF. Its role is profile selection and domain separation: members that disagree on the Shield policy compute a different key schedule and fail confirmation instead of silently sharing a downgraded epoch.

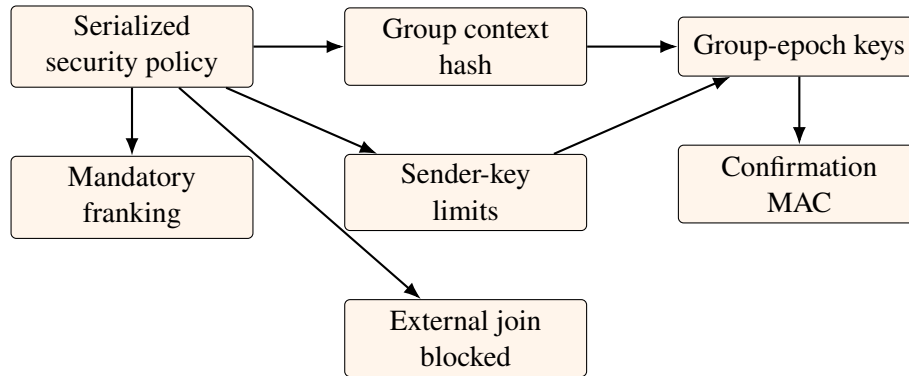


Figure 4: Shield as a cryptographic policy profile. Policy bytes affect group-epoch derivation and also drive local message-processing limits.

5. Protocol Operations

This section gives state-transition pseudocode for the construction. It names the inputs that must be authenticated, the state fields that must be updated, and the checks that cause rejection. It is not a wire-format grammar; the operations are specified at the cryptographic acceptance boundary.

5.1. Group Creation and Admission

A group begins with a creator leaf, an initial policy, and a ratchet tree that contains the creator’s signed leaf material. Adding a member is a group-epoch transition, not a roster mutation performed by the relay. The new member must receive a Welcome, decrypt the joiner secret, rebuild the tree, verify the tree hash, parse the policy, derive group-epoch keys, and check the confirmation MAC before it treats itself as a member.

Algorithm 1: Create a policy-bound group and add one member.

Inputs. Creator identity I_A , joiner key package KP_B , and serialized policy $policy$.

Outputs. Updated creator state S'_A , Add Commit message com , and Welcome message wel .

1. **Create the genesis group state.** The creator builds the initial ratchet tree and derives the genesis context:

$$\begin{aligned} S_A &\leftarrow \text{CreateTree}(I_A, policy), \\ gid &\leftarrow S_A.gid, \quad treeHash_0 \leftarrow \text{TreeHash}(S_A.tree), \\ gch_0 &\leftarrow \text{GroupContext}(gid, 0, treeHash_0, policy), \\ (epochSecret_0, initSecret_0) &\leftarrow \text{InitEpoch}(gch_0). \end{aligned}$$

2. **Validate and insert the joining leaf.** Reject unless KP_B verifies as a signed key package. If it verifies, insert it into the tree and compute the hybrid update path:

$$\begin{aligned} T_1 &\leftarrow \text{InsertLeaf}(S_A.tree, KP_B), \\ (T_1, path, commitSecret_1) &\leftarrow \text{HybridUpdatePath}(T_1, S_A.leaf). \end{aligned}$$

3. **Derive the next group-epoch state.**

$$\begin{aligned} treeHash_1 &\leftarrow \text{TreeHash}(T_1), \\ gch_1 &\leftarrow \text{GroupContext}(gid, 1, treeHash_1, policy), \\ j_1 &\leftarrow \text{HKDF-Extract}(initSecret_0, commitSecret_1), \\ epochSecret_1 &\leftarrow \text{HKDF-Expand}(j_1, 32, \text{"Aura-Group-EpochSecret"} \parallel gch_1), \\ \left(\begin{array}{l} k_1^{\text{meta}}, k_1^{\text{welcome}} \\ k_1^{\text{confirm}}, initSecret_1 \end{array} \right) &\leftarrow \text{Subkeys}_{policy}(epochSecret_1), \\ \tau_1 &\leftarrow \text{HMAC}(k_1^{\text{confirm}}, gch_1). \end{aligned}$$

4. **Produce the authenticated admission messages.**

$$\begin{aligned} com &\leftarrow \text{Sign}_{I_A}(gid, 0, 1, \text{Add}(KP_B), path, gch_1, \tau_1), \\ wel &\leftarrow \text{SealWelcome}_{KP_B}(gid, 1, j_1, T_1, treeHash_1, policy, \tau_1). \end{aligned}$$

5. **Install the creator's accepted group epoch.**

$$\begin{aligned} S'_A &\leftarrow \text{InstallEpoch}(S_A, T_1, 1, policy, epochSecret_1, initSecret_1), \\ S'_A.senderChains &\leftarrow \text{ResetSenderChains}(epochSecret_1, S'_A.members). \end{aligned}$$

Return (S'_A, com, wel) .

Algorithm 2: Process a Welcome as the joining member.

Inputs. Welcome message wel , local joiner secret-key material, and expected identity I_B .

Output. Accepted member state S_B , or rejection.

1. Open Welcome and parse policy. The joiner decrypts wel under its leaf credentials. It rejects if opening fails or if the serialized policy cannot be parsed.

2. Reconstruct the public group state. From the Welcome information, the joiner rebuilds the ratchet tree T and checks that I_B has a leaf, that the roster verifies, and that

$$\text{TreeHash}(T) = \text{info.treeHash}.$$

3. Recompute the policy-bound context and subkeys.

$$\begin{aligned} gch &\leftarrow \text{GroupContext}(\text{info.groupId}, \text{info.epoch}, \text{TreeHash}(T), \text{policy}), \\ epochSecret &\leftarrow \text{DeriveEpochSecret}(\text{info.joinerSecret}, gch), \\ \left(\begin{array}{l} k^{\text{meta}}, k^{\text{welcome}} \\ k^{\text{confirm}}, \text{initSecret} \end{array} \right) &\leftarrow \text{Subkeys}_{\text{policy}}(epochSecret). \end{aligned}$$

4. Verify confirmation and install state. Reject unless

$$\text{HMAC}(k^{\text{confirm}}, gch) = \text{info.confirmation}.$$

On success, install the accepted group epoch and reset sender chains:

$$S_B \leftarrow \text{InstallEpoch}(T, \text{info.groupId}, \text{info.epoch}, \text{policy}, epochSecret, \text{initSecret}).$$

The sender-chain bases of S_B are reset from $epochSecret$ for the accepted roster. Return S_B .

5.2. Group-Epoch Transition Processing

An Update, Add, Remove, PSK, or ReInit proposal becomes authoritative only after a signed Commit message is accepted. For the UpdatePath-bearing Commit messages analyzed here, the Commit identifies the old group epoch, the new group epoch, the proposal set, the committer leaf, the update path, and the confirmation MAC. Members process the update path from their own position in the tree: they locate the first path secret encrypted to a node in their resolution, decrypt it with the hybrid wrapper, derive the remaining path nodes, verify parent hashes, and then derive the group-epoch keys. PSK-only and ReInit-only Commit messages use the same context and confirmation checks but obtain their transition secret from proposal processing rather than from DecryptOrApplyUpdatePath.

Algorithm 3: Process an UpdatePath-bearing group-epoch Commit message.

Inputs. Local state S and signed Commit message com .

Output. New accepted state S' , removal notification, or rejection.

1. **Check the public transition envelope.** Reject unless the Commit is for $S.\text{gid}$, advances from $S.\text{epoch}$ to $S.\text{epoch} + 1$, carries the same policy bytes as $S.\text{policy}$, has a valid committer signature, and contains proposals valid for the current roster.
2. **Apply public tree changes.** Apply the public proposal effects to obtain a candidate tree T . If the local leaf is removed, erase local group-epoch secrets and return Removed.
3. **Recover and authenticate the update path.** From the local leaf position, decrypt or apply the first reachable path secret:

$$(T, \text{commitSecret}) \leftarrow \text{DecryptOrApplyUpdatePath}(T, \text{com.updatePath}, S.\text{leaf}).$$

Reject if no path secret can be recovered or if the parent hashes do not verify against com.committer .

4. **Recompute the accepted group context.**

$$gch \leftarrow \text{GroupContext}(S.\text{gid}, S.\text{epoch} + 1, \text{TreeHash}(T), S.\text{policy}).$$

Reject unless $\text{com.context} = gch$.

5. **Derive the new group-epoch keys.**

$$\begin{aligned} j &\leftarrow \text{HKDF-Extract}(S.\text{initSecret}, \text{commitSecret}), \\ \text{epochSecret} &\leftarrow \text{HKDF-Expand}(j, 32, \text{"Aura-Group-EpochSecret"} \parallel gch), \\ \left(\begin{array}{l} k^{\text{meta}}, k^{\text{welcome}} \\ k^{\text{confirm}}, \text{initSecret} \end{array} \right) &\leftarrow \text{Subkeys}_{S.\text{policy}}(\text{epochSecret}). \end{aligned}$$

Reject unless $\text{HMAC}(k^{\text{confirm}}, gch) = \text{com.confirmation}$.

6. **Install the new group epoch atomically.**

$$\begin{aligned} S' &\leftarrow \text{InstallEpoch}(S, T, S.\text{epoch} + 1, S.\text{policy}, \text{epochSecret}, \text{initSecret}), \\ S'.\text{senderChains} &\leftarrow \text{ResetSenderChains}(\text{epochSecret}, S'.\text{members}). \end{aligned}$$

Return S' .

5.3. *Application Messages*

Application messages have lower computational and bandwidth cost than group-epoch transition Commit messages. The sender advances its own chain, encrypts sender data under the group-epoch metadata key, encrypts payload content under the message key, signs the envelope, and optionally attaches a franking tag. The receiver authenticates the signature, decrypts sender data, checks the replay window, derives or retrieves the sender-chain key, decrypts the payload, and then applies policy checks such as mandatory franking, expiration, maximum size, and allowed message semantics. The sender and receiver state updates are monotone: a sent message persists the advanced sender chain, and an accepted receive persists replay-window state before plaintext is exposed to the application.

Algorithm 4: Encrypt and decrypt a group-epoch application message.

Inputs. Sender state S_i , receiver state S_j , and plaintext m .

Outputs. Updated sender state S'_i , ciphertext C , and either updated receiver state S'_j with plaintext m' or rejection.

1. Sender advances its chain and prepares sender data.

$$\begin{aligned}
 (mk, ck'_i) &\leftarrow \text{AdvanceSenderChain}(S_i.ck_i), \\
 (n_{sd}, n_{pt}, reuseGuard) &\leftarrow \text{FreshMessageRandomness}(), \\
 sd &\leftarrow (i, S_i.generation, reuseGuard), \\
 aad_{sd} &\leftarrow S_i.gid \parallel S_i.epoch, \\
 C_{sd} &\leftarrow \text{AES-256-GCM-SIV.Enc}(S_i.k^{\text{meta}}, n_{sd}, sd, aad_{sd}).
 \end{aligned}$$

2. Sender applies message semantics and encrypts the payload. If franking is required or requested, compute $(fk, tag) \leftarrow \text{Frank}(m)$ and set $m^* \leftarrow (m, fk)$; otherwise set $tag \leftarrow \varepsilon$ and $m^* \leftarrow m$. Then compute

$$\begin{aligned}
 aad_{pt} &\leftarrow aad_{sd} \parallel C_{sd} \parallel tag, \\
 C_{pt} &\leftarrow \text{AES-256-GCM-SIV.Enc}(mk, n_{pt}, m^*, aad_{pt}), \\
 \sigma &\leftarrow \text{Sign}_{I_i}(S_i.gid, S_i.epoch, n_{sd}, n_{pt}, C_{sd}, C_{pt}, tag), \\
 C &\leftarrow (S_i.gid, S_i.epoch, n_{sd}, n_{pt}, C_{sd}, C_{pt}, tag, \sigma).
 \end{aligned}$$

Set $S'_i \leftarrow S_i[ck_i \leftarrow ck'_i, generation \leftarrow S_i.generation + 1]$ and persist S'_i before releasing C .

3. Receiver authenticates the envelope and opens sender data. Reject unless C matches the receiver's group identifier and group epoch and the sender signature verifies for the current roster. Then compute

$$sd' \leftarrow \text{AES-256-GCM-SIV.Dec}(S_j.k^{\text{meta}}, n_{sd}, C_{sd}, S_j.gid \parallel S_j.epoch).$$

Parse sd' as $(i', g', reuseGuard')$ and set $rk'_{\text{replay}} \leftarrow (i', g')$. Reject if sender-data decryption fails, if the sender leaf or generation is malformed for the current roster, or if rk'_{replay} is already present in the replay window.

4. Receiver resolves the sender-chain key and decrypts the payload.

$$\begin{aligned}
 (mk', S_j^*) &\leftarrow \text{ResolveSenderKey}(S_j, sd'), \\
 m^{*'} &\leftarrow \text{AES-256-GCM-SIV.Dec}(mk', n_{pt}, C_{pt}, S_j.gid \parallel S_j.epoch \parallel C_{sd} \parallel tag).
 \end{aligned}$$

Reject if key resolution or payload decryption fails.

5. Receiver applies policy and commits receive state. Open the payload semantics to obtain m' , reject unless $\text{PolicyAccepts}(S_j.policy, m^{*'}, tag)$ holds. Set $S'_j \leftarrow \text{CommitReceiveState}(S_j^*, sd')$, which marks rk'_{replay} as seen, and persist S'_j before exposing m' . Return (S'_i, C, S'_j, m') .

6. Message Layer

TreeKEM Commit messages are comparatively expensive and are used for group-epoch changes. Inside one group epoch, application messages use sender chains. At group-epoch installation the base sender chain for leaf i is reset from the accepted group-epoch secret; for sender leaf i and generation g :

$$ck_{i,0}^e = \text{HKDF-Expand}(\text{epochSecret}_e, 32, \text{“Aura-Group-SenderChain”} \parallel gch_e \parallel \text{le}_{32}(i)), \quad (11)$$

$$mk_{i,g} = \text{HKDF-Expand}(ck_{i,g}, 32, \text{“Aura-Group-Msg”}), \quad (12)$$

$$\widetilde{ck}_{i,g+1} = \text{HKDF-Expand}(ck_{i,g}, 32, \text{“Aura-Group-Chain”}), \quad (13)$$

$$ck_{i,g+1} = \begin{cases} \widetilde{ck}_{i,g+1}, & \text{default policy,} \\ \text{BLAKE2bMAC}^{\text{“Aura-B2Chain”}}(\widetilde{ck}_{i,g+1}), & \text{Shield.} \end{cases} \quad (14)$$

The payload is padded and encrypted under $mk_{i,g}$. Sender metadata $(i, g, \text{reuseGuard})$ is encrypted under the group-epoch metadata key k_e^{meta} . The message signature covers the group identifier, group-epoch number, encrypted sender data, encrypted payload, nonces, and franking tag.

6.1. Replay Rejection

Each sender has a replay window with a high-water generation and a bitmap. A duplicate generation inside the window is rejected. A generation too far behind the window is also rejected. The skipped-key cache is bounded per sender and globally; Shield tightens the per-sender bound.

6.2. Franking, Sealed Content, and Expiration

For a frankable message the sender samples a franking key fk and publishes:

$$\text{tag} = \begin{cases} \text{HMAC}(fk, \text{content} \parallel \text{sealedContent}), & \text{frankable,} \\ \varepsilon, & \text{otherwise.} \end{cases} \quad (15)$$

The tag is visible in the application message, while fk is inside the end-to-end encrypted payload. A recipient who reports a message can disclose $(fk, \text{content}, \text{sealedContent}, \text{tag})$ for that message only. The relay cannot verify or read unreported messages because it does not know fk ; this follows the message franking goal of selective accountable disclosure [17].

Sealed content places an encrypted inner payload behind a visible hint, in the same relay-blind spirit as sealed-sender metadata minimization [18]. The seal key

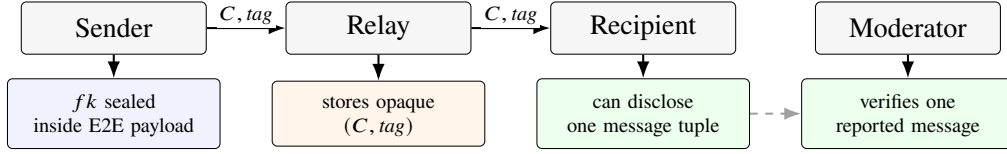


Figure 5: Franking disclosure path. The relay stores an opaque commitment, while a recipient can later reveal exactly one message to a moderator.

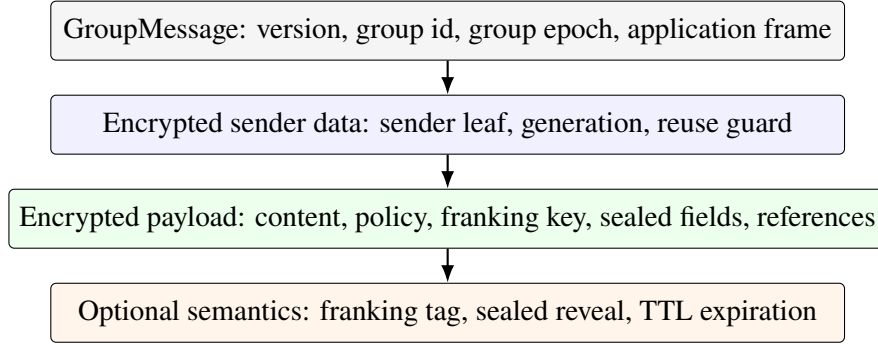


Figure 6: Application-message envelope. Sender data is hidden from passive observers under the group-epoch metadata key, while the payload uses the sender-chain message key.

is derived from the message key, and the recipient obtains it only after group decryption. Disappearing messages carry a send timestamp and TTL; decryption rejects expired messages and timestamps too far in the future.

7. External Join and Relay Model

External join is supported only when policy permits it. The joining member uses external key material derived from the current init secret and presents an authorization signed by an existing member. The authorization binds the group identifier, group-epoch number, group context hash, external public keys, authorizer leaf, joiner identity, and expiry. In Shield groups this path is disabled.

The authorization is evaluated against the public state and group context for the group epoch in which it was issued. A joiner cannot transplant it to a different tree hash, policy, group identifier, external key pair, or later group epoch without changing the signed data. Expiry is an authorization-creation boundary: it prevents a stale authorization from being used to form a new external-join Commit message. Once a valid external-join Commit message has been formed for the bound group epoch, offline members may still process that Commit later, because their acceptance check is the signed Commit message and policy-bound group-epoch transition rather than the joiner’s local wall clock.

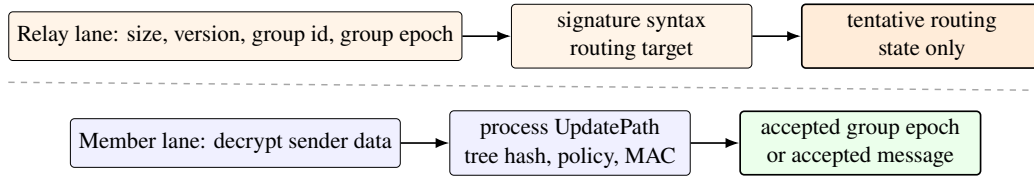


Figure 7: Relay/member authority split. Relay checks are structural and useful for routing, while cryptographic acceptance remains a member-side state transition.

Table 5: Relay-visible checks versus member-side checks.

Check	Relay side	Member side
Envelope format	Version, size, group identifier, group-epoch number, payload type.	Same, plus decryption under local group-epoch keys.
Commit structure	Group-epoch increment, group identifier, syntactic proposal shape, signature binding.	Proposal validity, TreeKEM path processing, tree hash, confirmation MAC.
Application message	Sender identity binding from authenticated context, signature syntax, routing target.	Sender-data decrypt, replay window, sender-chain key, AEAD payload decrypt, policy checks.
Membership state	Tentative routing roster.	Authoritative ratchet-tree state and accepted group epoch.

The relay model is narrower. The relay may validate envelope structure, group identifier, group-epoch number, sender signature format, and routing information. For Commit messages it can maintain tentative delivery state, but it cannot verify the TreeKEM path secret, confirmation MAC, or final client-side membership state because it has no group secrets. This separation is important: relay-side validation is useful for abuse resistance and routing, but it is not a cryptographic authority.

8. Security Argument

The security argument is modular rather than fully composable. It separates the cryptographic assumptions used by individual mechanisms from the protocol invariants that connect those mechanisms inside one accepted group epoch. This is a defined claim boundary: the argument does not prove arbitrary asynchronous group schedules. It argues that, for a locally accepted transition, the tree update, policy context, confirmation MAC, sender-chain state, replay window, and message semantics fit together in the intended way.

Table 6: Assumption use in the informal security argument.

Mechanism	Assumption or invariant	Used for
X25519 path component	Classical DH hardness before a CRQC and fresh post-compromise keys.	Hybrid path secrecy and local recovery when the classical component survives.
ML-KEM path component	IND-CCA2 security of ML-KEM-768.	Path secrecy against harvest-now-decrypt-later when the KEM component survives.
HKDF schedule	Extractor/PRF behavior with domain-separated labels.	Combining previous init, commit secret, group-epoch context, and sender-chain derivation.
AES-256-GCM-SIV	Authenticated encryption with misuse resistance under bounded nonce misuse.	Payload, sender-data, Welcome, and sealed-state confidentiality/integrity.
Ed25519 signatures	Unforgeability of identity, Commit-message, and message signatures.	Key-package authenticity, Commit origin, and sender binding.
Local erasure	Old path and group-epoch secrets are erased; any retained delayed-message keys are bounded and tagged by group epoch.	Forward secrecy, delayed delivery, and post-compromise recovery boundaries.
State continuity	Commit/message acceptance is atomic with durable replay and sealed-state updates.	Preventing rollback from turning old ciphertexts into fresh events.

The central invariant is that there is a single accepted context for a member at a group epoch: the tuple $(gid, e, treeHash_e, policy)$ determines gch_e , and gch_e determines the group-epoch keys and confirmation MAC. If any component changes, honest clients either derive different keys or fail confirmation. The second invariant is membership-indexed key availability: only leaves represented in the accepted ratchet tree, and not removed by the accepted Commit, obtain the path secret needed for the next group epoch. The third invariant is sender-indexed message state: once a group epoch is accepted, each sender has an independent chain and replay window, so duplicate generations do not become fresh messages. The fourth invariant is state continuity: rejected Commit messages and rejected ciphertexts

do not partially advance group-epoch counters, sender chains, replay windows, or sealed-state freshness markers.

The positive claims below should therefore be read as one-step accepted-state claims. They are not global liveness claims about delivery, deniability claims, or a universal-composability theorem for arbitrary group schedules. Within that boundary, the argument identifies the exact state transition at which fresh hybrid path material, policy consensus, and sender-chain reset enter the protocol state.

Claim 8.1 (Hybrid group-epoch recovery). *Assume honest erasure of old path secrets and group-epoch secrets. If an accepted Commit after compromise contains fresh X25519 and ML-KEM-768 material generated by an uncompromised member, then the new group-epoch secret is pseudorandom provided at least one of the hybrid wrapping components remains secure and HKDF behaves as a randomness extractor and PRF.*

The argument follows the hybrid-combiner principle. Path secrets are delivered under a wrapping key derived from both a X25519 shared value and a ML-KEM-768 shared secret. Replacing one surviving component by random changes the wrapping key to a pseudorandom key. The commit secret then feeds HKDF-Extract with the previous init secret; the group-epoch context binds the accepted tree and policy. If the previous init secret was compromised, recovery depends on the fresh commit secret. If the update path was generated before compromise or by a compromised member, this claim does not apply. Thus the recovery point is the first accepted honest group-epoch update whose path material was sampled after the compromise event.

Claim 8.2 (Policy consensus). *Two members that accept the same Commit but use different serialized security policies derive different group context hashes except with collision probability for SHA-256. Consequently, they derive different group-epoch keys and fail confirmation or later AEAD checks.*

This claim is the core reason to treat policy as cryptographic input. A relay or application cannot silently downgrade Shield to a weaker policy while preserving the same group-epoch transcript. The same reasoning covers sender-key limits, mandatory franking, and external join policy because those fields are serialized into the policy bytes before hashing. A member that accepts the weaker policy enters a different group-epoch state from a member that accepts the stronger one; the mismatch is detected by confirmation or by later message decryption failure.

Claim 8.3 (Removed-member exclusion). *Let member R be removed by an accepted Commit from group epoch e to $e + 1$. If R does not receive a valid path secret for group epoch $e + 1$ and the remaining members erase old group-epoch keys and message keys as specified, then R cannot derive sender-chain message keys for group epoch $e + 1$ from its old group-epoch- e state.*

The public Commit may reveal the proposal that removes R , but the new group-epoch secret is derived from the previous init secret and the fresh commit secret recovered through the update path. The update path is addressed to the co-path resolution of current recipients, not to removed leaves. Sender-chain bases are reset from the new group-epoch secret. Consequently, old sender-chain state does not extend across the removal boundary. This is a membership-interval exclusion statement, not a claim that already-delivered group-epoch- e ciphertexts become undecryptable.

If R is later added again, the admission is a new group-epoch transition with a new key package or external-join authorization, a new leaf position as accepted by the tree, and fresh sender-chain bases derived from the re-addition group epoch. The new membership interval does not bridge the removal gap. In particular, successful re-addition gives R access to group epochs at and after the new Welcome or accepted external join, not to application messages from group epochs in which R was absent.

Claim 8.4 (Replay rejection). *Within one group epoch, a valid application ciphertext with the same sender leaf and generation is accepted at most once by a receiver that preserves its replay window state.*

The receiver decrypts sender data under the group-epoch metadata key, verifies the sender signature, checks the replay window, obtains the sender-chain message key, and only then marks the generation as seen. Modified replays fail at signature, AEAD, franking, or replay checks depending on the mutation. Persistence matters: if a receiver restores old sealed state without an external anti-rollback floor, replay windows may also roll back. The security claim therefore requires sealed-state anti-rollback metadata to be treated as deployment state rather than as a cosmetic serialization field.

Claim 8.5 (Delayed-message state separation). *If delayed application messages from previous group epochs are supported, the retained compatibility state is bounded, tagged by group epoch, and message-key scoped. Accepting such a delayed message cannot roll the current group epoch backward, reinstall an old tree, or revive a removed sender-chain state.*

This separates asynchronous delivery from rollback. A member may keep a small cache of skipped message keys or metadata-opening material for messages that were already authorized in their source group epoch. That cache is not the previous group-epoch secret, not the previous init secret, and not authority to process new Commit messages under an old tree. The message is checked against the group identifier, source group epoch, sender leaf, generation, replay window, and policy semantics for the group epoch that produced the key. If the cache is absent or exhausted, the message is rejected; the implementation does not synthesize old state from the current group epoch.

Claim 8.6 (Franking accountability). *If a moderator verifies a disclosed tuple $(fk, content, sealedContent, tag)$ for an accepted frankable message, then the tag corresponds to the franking key and content fields carried in an end-to-end encrypted payload accepted by a group member, except with the forgery probability of the MAC.*

The relay sees the franking tag before any report, but it does not know the franking key because fk is carried inside the encrypted payload. A recipient can disclose fk , the relevant content, the sealed-content field, and the tag for one message. The moderator then checks the MAC relation for that message only. This does not give the moderator keys for unrelated messages, and it does not let the relay verify ordinary unreported traffic.

8.1. Failure Cases Outside the Claim Boundary

Several important failure cases are outside the positive claims. If all live members that can update the tree are compromised and later no honest member performs an update, there is no post-compromise recovery event. If clients accept unsigned or incorrectly bound key packages, then group membership authentication has already failed before TreeKEM processing begins. If a deployment permits policy changes without treating them as explicit Commit messages, then the policy-consensus argument no longer applies. If sealed state can be rolled back by the local platform, replay windows and skipped-key budgets can also be rolled back. These conditions are stated as part of the interface contract for the protocol state machine.

9. Scoped Symbolic Model

The construction is accompanied by the scoped ProVerif model formal/proverif/aura_group.pv [19]. It can be run with `make -C formal proverif-group`. The model covers one accepted hybrid update path and one application

Table 7: Verification scope and non-claims for AURA Group.

Evidence source	In scope	Explicitly out of scope
Protocol argument	Accepted epoch secrecy, hybrid recovery boundary, policy consensus, removed-member exclusion, replay rejection, delayed-message separation, and franking accountability.	Universal composability for all asynchronous group schedules, delivery fairness, and legal or product moderation semantics.
ProVerif model	One accepted hybrid update path, one resulting group epoch, one application message, and six secrecy/correspondence queries.	Full ratchet-tree traversal, all co-path recipients, proposal queues, skipped-key caches, replay bitmaps, and multiple concurrent senders.
Reference implementation	Parsing, rejection paths, transactional state movement, persistence, replay windows, external-join authorization, Shield policy enforcement, franking, sealed state, and benchmarked cost centers.	A substitute for reduction-style proof, a proof of resource-exhaustion resistance, or evidence for unimplemented deployment policies.
Benchmarks	Cost characterization for group creation, admission, application messages, group updates, and sealed-state import/export.	Security assumptions, production capacity guarantees, or mobile-device performance claims.

message inside the resulting group epoch. The network adversary controls the public channel, can replay and substitute public messages, and sees the hybrid path wrapper and application ciphertext; honest acceptance is represented by confirmation-MAC and correspondence events. The model abstracts away full asynchronous group scheduling, proposal queues, delivery-service fairness, multiple concurrent senders, and multi-device state.

The six queries check the two composition points that the construction relies on: the Commit-to-epoch transition, where a path secret becomes a group-epoch secret under a policy-bound context, and the epoch-to-message transition, where the accepted epoch key becomes a message and franking key. The model represents the hybrid path wrapper, the policy-bound confirmation MAC, the application payload, and franking correspondence explicitly. Full tree traversal, all co-path recipients, skipped-key cache behavior, replay bitmaps, and arbitrary proposal interleavings remain outside this one-epoch model. This is a scoped positive result: the model tests the intended cryptographic boundary rather than replacing a future concurrent TreeKEM proof.

Table 8: ProVerif group model: scoped queries discharged.

Query	Model event or secrecy target	Result	Meaning
Q1	path_secret secrecy	true	The fresh TreeKEM path secret is not learned from the hybrid path wrapper.
Q2	prev_init_secret secrecy	true	The previous init-secret contribution remains hidden in the accepted transition.
Q3	app_secret secrecy	true	The challenge payload within the accepted group epoch is not learned by the network adversary.
Q4	CommitAccepted CommitCreated	$\Rightarrow$ true	An accepted Commit corresponds to an honest policy-bound Commit event.
Q5	MessageAccepted MessageSent	$\Rightarrow$ true	An accepted challenge message corresponds to an honest send under the same group-epoch key.
Q6	FrankingVerified FrankingIssued	$\Rightarrow$ true	A verified franking disclosure corresponds to an issued franking tag.

10. Reference Implementation Validation

The reference-implementation validation evidence is summarized in Table 9. It is organized by validation class: lifecycle transitions, member exclusion, external-join authorization, message-layer misuse cases, higher-level message semantics, sealed-state persistence, and cost characterization. These checks show that the executable state machine enforces the same acceptance and rejection boundaries described by the specification.

The validation suite exercises the construction at several layers: external interface behavior, lower-level TreeKEM invariants, key-package validation, key-schedule domain separation, message-layer misuse cases, time-dependent semantics, persistence checks, and adversarial rejection paths. The evidence is reported by state-machine coverage: the executable transitions are checked against the same acceptance and rejection boundaries specified by the construction.

The validation evidence covers hybrid path-secret processing, parent-hash val-

Table 9: Reference-implementation validation evidence for the group construction.

Area	Evidence	Security relevance
TreeKEM update	Add, remove, update, interleaving, and parent-hash validation cases.	Group-epoch state evolves through authenticated update paths.
Removed member exclusion	Rejection of post-removal decrypt attempts and stale captured ciphertexts.	Old members do not obtain post-removal group-epoch keys.
External join	Rejection of stale, expired, tampered, wrong-state, or policy-blocked join attempts.	Join surface is explicit and policy controlled.
Message layer	Replay, ciphertext-tamper, and sender-authentication rejection cases.	Application messages are bound to group epoch, sender, and generation.
Message semantics	Validation of sealed, expiring, frankable, edited, and deleted message semantics.	Higher-level semantics are enforced after cryptographic decrypt.
Sealed state	Rejection of malformed, wrong-key, wrong-identity, tampered, and rolled-back state.	Local state persistence has integrity and external freshness requirements.
Performance	Measurements of group creation, admission, message processing, updates, and state sealing.	Cost measurements are engineering context, not security assumptions.

idation, path-key rotation, pre-join and post-removal confidentiality boundaries, re-addition gap protection, per-group-epoch replay rejection, external-join authorization binding, Shield policy enforcement, franking tamper rejection, and sealed-state integrity. This evidence supports the protocol description at the engineering boundary where parsing, persistence, replay handling, and rejection behavior are observable.

Benchmarks are reported separately from security evidence. They measure the principal cost centers of the construction: creating a group, admitting a member, encrypting and decrypting group messages, performing an update, and exporting or importing sealed group state. Benchmark timings are not used as assumptions; the measurements are engineering context.

10.1. Reproducibility Notes

A reviewer can examine the construction through four views: the protocol description, the reference implementation, the validation corpus, and the symbolic model. The manuscript gives the protocol-level equations and claim boundary; the implementation realizes tree state, TreeKEM wrapping, Commit messages, key packages, Welcome processing, sender keys, and key schedule derivation; the validation material and symbolic model exercise these layers from complementary perspectives. Benchmark tables and validation counts are included for reproducibility and engineering context.

The artifact workflow records the exact source revision, operating system, CPU, Rust toolchain, dependency lockfile, dirty-worktree state, validation commands, formal-model logs, and benchmark summaries. This is the intended reproducibility boundary: a reviewer can rerun the symbolic model, validation suite, and group benchmark targets, but the resulting logs remain engineering evidence rather than additional cryptographic claims.

11. Comparison

11.1. Interpretation

The comparison in Table 10 positions the protocol rather than ranking systems on a single axis. MLS is the mature standard reference for TreeKEM-based group messaging. Sender-key style designs are efficient for application traffic. AURA Group combines those design principles in a narrower profile: use a tree for epoch-changing membership updates, carry hybrid X25519+ML-KEM-768 material in the update path, and bind sender-chain message processing to the accepted epoch policy.

The main tradeoff is complexity at the group-epoch boundary. A hybrid update path must carry more public material and more ciphertext material than a purely classical update. It also requires careful binding between recipient resolution, node index, tree hash, and policy context. The benefit is that a fresh honest group-epoch update can incorporate post-quantum material into the group state itself, rather than confining the post-quantum contribution to admission.

The policy-binding design also has a deployment cost. If a product wants mutable policy, such as toggling franking or changing skipped-key limits, that policy change must be represented as an explicit group-epoch transition. This is less convenient than a server-side flag, but it yields an explicit cryptographic downgrade boundary for protocol participants. In high-risk groups that boundary is preferable:

Table 10: Design positioning against common group-messaging design points.

Property		MLS 1.0 baseline	Sender-key style groups	AURA Group profile
Group-epoch structure		Standardized TreeKEM epochs and group context in RFC 9420.	Often pairwise admission plus symmetric sender-key traffic.	MLS-inspired epochs with hybrid TreeKEM update-path wrapping.
Post-quantum material		Not specified by the RFC 9420 baseline.	Depends on the system’s pairwise setup and update policy.	X25519+ML-KEM-768 material carried in membership-changing update paths.
Policy as key input		Group context is central; application policy profiles are separate.	Commonly enforced above the cryptographic group state.	Serialized policy is hashed into the epoch context and subkey schedule.
Within-epoch messages	mes-	Application secrets derive from the accepted epoch state.	Efficient sender chains.	Per-member sender chains with encrypted sender data and epoch binding.
Accountability hooks		Not the main protocol goal.	Product-specific.	Franking and sealed-content hooks bound to accepted epoch policy.
Relay authority		Delivery service is not a group-secret holder.	Service role varies by system.	Relay performs structural checks; clients decide cryptographic acceptance.

policy is accepted when the group epoch is accepted, not when a relay says it has changed.

The scientific point is therefore not the invention of TreeKEM, sender keys, or message franking in isolation. It is the composition boundary for a concrete post-quantum group profile: a single accepted group epoch determines tree state, policy, metadata protection, sender-chain bases, replay state, and accountable message semantics. This is the boundary that the equations, algorithms, symbolic model, and implementation evidence are organized to inspect.

12. Limitations

Table 11: Ranked limitations and deployment effect.

Severity	Limitation	Effect on the claim boundary
High	Classical identity and Commit signatures	Identity and Commit-message authentication use Ed25519. Hybrid TreeKEM secrecy is post-quantum only for the key-establishment material; migration to ML-DSA or another post-quantum signature scheme is separate work.
High	Concurrent group proof boundary	The ProVerif model covers one accepted group-epoch transition and one application message. Complete group-security claims require explicit tree state, proposal queues, external joins, replay windows, and concurrency at the level expected in formal MLS analyses [11, 20].
High	Platform rollback and delivery fairness	The protocol rejects stale or malformed state, but it cannot force relay delivery or prevent a platform from rolling back local sealed state without an external freshness floor. Replay and delayed-message claims therefore depend on durable state continuity.
Medium	Single-device model	Multi-device fan-out, per-device membership, device churn, and cross-device state repair require a separate membership and recovery model.
Medium	Relay validation is structural	The relay can enforce version, size, routing, and syntactic checks, but it cannot validate TreeKEM secrets or act as authoritative group membership state.
Medium	Mutable policy requires explicit Commit semantics	The current construction treats policy as part of accepted group state. Dynamic policy changes require protocol-level Commit semantics and downgrade analysis, not a server-side flag.
Medium	No deniability proof	Message signatures and franking support sender binding and accountable disclosure. The manuscript does not claim deniable authentication for group application messages.
Medium	No full resource-exhaustion proof	The implementation bounds parsing, skipped-key storage, and replay windows, but adversarial memory/CPU exhaustion under hostile traffic is an engineering analysis outside the present model.

13. Conclusion

AURA Group specifies a compact hybrid group-messaging protocol in which membership updates, policy agreement, and application-message processing are all anchored at the accepted group epoch. The central design choice is to put post-quantum material in ordinary TreeKEM update paths and to put serialized security

policy in the group context that drives key derivation and confirmation. As a result, a membership-changing Commit does more than update the tree: it installs the cryptographic interval in which sender chains, metadata protection, Shield processing, franking, sealed state, replay windows, and expiration checks are interpreted.

The paper gives an inspectable design point and a stated evidence boundary for post-quantum confidential group state. The informal argument covers hybrid epoch recovery, policy consensus, removed-member exclusion, replay rejection, delayed-message separation, and franking accountability under explicit erasure and state-continuity assumptions. The scoped ProVerif model checks six queries for one accepted update path and one application message, and the implementation validation exercises the parser, state machine, rejection paths, external-join controls, Shield policy, franking, sealed persistence, and benchmark cost centers. The result is a protocol-level account of how post-quantum path material and application-security policy enter the same accepted group state, with the remaining concurrency and deployment obligations made explicit rather than hidden in the implementation.

The remaining work is correspondingly clear: post-quantum authentication, a concurrent tree proof for arbitrary proposal schedules, multi-device group state, delivery-service fairness, platform anti-rollback anchors, mobile-class benchmarks, and hostile-traffic resource analysis. Within the scope stated here, AURA Group offers a concrete protocol construction for policy-bound hybrid TreeKEM epochs and efficient group messages under an auditable post-quantum confidentiality boundary.

Data and Software Availability

The source code, validation material, symbolic models, benchmark summaries, reproducibility scripts, and manuscript sources are maintained in the accompanying source repository: <https://github.com/oleksandrmelnynchenko/aura-protected-protocol-rs>. The accompanying artifact contains the exact source snapshot used for the implementation, symbolic-model, validation, and benchmark results, together with the generated versions.txt, MANIFEST.txt, SHA256SUMS, formal-model logs, benchmark summaries, and validation outputs. These files are the reproducibility record for the results in this manuscript; later repository revisions are treated as continuing development rather than as the cited artifact.

Funding

The author declares that this research received no external funding.

CRedit Author Statement

Oleksandr Melnychenko: conceptualization, methodology, software, validation, symbolic modeling, investigation, writing—original draft, writing—review and editing, and visualization.

Declaration of Competing Interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported here.

Declaration of Generative AI and AI-assisted technologies in the writing process

OpenAI Codex was used during manuscript preparation to assist with English-language editing, consistency checks, and minor L^AT_EX typesetting support. OpenAI Codex was not used as an authoritative source for technical claims, references, equations, implementation results, or experimental measurements. All protocol descriptions, equations, references, tables, implementation details, and experimental results were independently reviewed, verified, and approved by the author, who takes full responsibility for the content of the article.

References

- [1] O. Melnychenko. Aura Protocol: Hybrid Post-Quantum Confidentiality for Secure Messaging via Per-Ratchet KEM Rotation. Companion manuscript, 2026. Manuscript under preparation.
- [2] O. Melnychenko. Hybrid Post-Quantum Authentication from OPAQUE and ML-KEM-768: Construction, Formal Analysis, and a Reproducible Implementation. Companion manuscript, 2026. Manuscript under preparation.
- [3] O. Melnychenko. Formal Verification Report—Hybrid PQ-OPAQUE Protocol. Companion formal-evidence report, 2026. Technical report under preparation.
- [4] S. Jarecki, H. Krawczyk, and J. Xu. OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. In *EUROCRYPT 2018*, pages 456–486. Springer, 2018. https://doi.org/10.1007/978-3-319-78372-7_15

- [5] D. Bourdrez, H. Krawczyk, K. Lewi, and C. A. Wood. The OPAQUE Augmented Password-Authenticated Key Exchange (aPAKE) Protocol. RFC 9807, 2025. <https://www.rfc-editor.org/info/rfc9807>
- [6] S. Svystun, L. Scisło, M. Pawlik, O. Melnychenko, P. Radiuk, O. Savenko, and A. Sachenko. DyTAM: Accelerating wind turbine inspections with dynamic UAV trajectory adaptation. *Energies*, 18(7):1823, 2025. <https://api.crossref.org/works/10.3390/en18071823>
- [7] S. Svystun, O. Melnychenko, P. Radiuk, O. Savenko, A. Sachenko, and A. Lysyi. Thermal and RGB images work better together in wind turbine damage detection. *International Journal of Computing*, 23(4):526–535, 2024. <https://doi.org/10.47839/ijc.23.4.3752>
- [8] A. Lysyi, A. Sachenko, P. Radiuk, M. Lysyi, O. Melnychenko, O. Ishchuk, and O. Savenko. Enhanced fire hazard detection in solar power plants: an integrated UAV, AI, and SCADA-based approach. *Radioelectronic and Computer Systems*, (2):99–117, 2025. <https://doi.org/10.32620/reks.2025.2.06>
- [9] R. Barnes, B. Beurdouche, R. Robert, J. Millican, E. Omara, and K. Cohn-Gordon. The Messaging Layer Security (MLS) Protocol. RFC 9420, 2023. <https://www.rfc-editor.org/rfc/rfc9420>
- [10] K. Bhargavan, R. Barnes, and E. Rescorla. TreeKEM: Asynchronous Decentralized Key Management for Large Dynamic Groups. Protocol proposal for Messaging Layer Security, 2018. https://mailarchive.ietf.org/arch/msg/mls/v1CY0jFAOVHokB4DtNqS__tX1o/4/
- [11] T. Wallez, J. Protzenko, B. Beurdouche, and K. Bhargavan. TreeSync: Authenticated Group Management for Messaging Layer Security. USENIX Security Symposium, 2023. <https://www.usenix.org/conference/usenixsecurity23/presentation/wallez>
- [12] National Institute of Standards and Technology. Post-Quantum Cryptography Standardization. <https://csrc.nist.gov/projects/post-quantum-cryptography>
- [13] National Institute of Standards and Technology. Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM). FIPS 203, 2024. <https://doi.org/10.6028/NIST.FIPS.203>
- [14] A. Langley, M. Hamburg, and S. Turner. Elliptic Curves for Security. RFC 7748, 2016. <https://www.rfc-editor.org/rfc/rfc7748>
- [15] H. Krawczyk and P. Eronen. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). RFC 5869, 2010. <https://www.rfc-editor.org/rfc/rfc5869>
- [16] A. Gueron, A. Langley, and Y. Lindell. AES-GCM-SIV: Nonce Misuse-

- Resistant Authenticated Encryption. RFC 8452, 2019. <https://www.rfc-editor.org/rfc/rfc8452>
- [17] P. Grubbs, J. Lu, and T. Ristenpart. Message franking via committing authenticated encryption. *Advances in Cryptology – CRYPTO 2017*. <https://eprint.iacr.org/2017/664>
 - [18] Signal. Technology preview: Sealed Sender for Signal. Signal Blog, 2018. <https://signal.org/blog/sealed-sender/>
 - [19] B. Blanchet. Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1–2):1–135, 2016. <https://bblanche.gitlabpages.inria.fr/publications/BlanchetFnTPS16.html>
 - [20] S. Meier, B. Schmidt, C. Cremers, and D. Basin. The TAMARIN prover for the symbolic analysis of security protocols. In *CAV 2013*, pages 696–701. Springer, 2013. https://doi.org/10.1007/978-3-642-39799-8_48